

Benchmark Report: pyflate

A small link budget, a compressed payload, and a lot of work between the bits

Matan Cohen · Yuval Kogan | Technion | HWSW Final Project

Somewhere between a spacecraft and a ground station there is a link budget measured in kilobits.

Compression makes the most of it; decompression turns the arriving bits back into useful data. This benchmark gives that quiet half of the job to Python: read a bit stream, recover symbols, and undo several transformations. Its Huffman matcher looks like the obvious place to start. Measuring it led us to improvements across the whole decoding pipeline.

1. Workload and result

Despite its name, the measured pyflate workload is **bzip2**, not DEFLATE. Each iteration decompresses `interpreter.tar.bz2` from **67,562 bytes to 399,360 bytes**; the harness checks the original MD5 outside the timer. The Python tier uses the standard library rather than calling a native decompression library. Its main structures are an integer bit buffer, Huffman tables, a move-to-front (MTF) alphabet, Burrows-Wheeler transform (BWT) index vectors and output buffers.

One block, six Huffman tables, 148,271 symbols. The decoder switches tables every 50 symbols. Huffman decoding, move-to-front and RUNA/RUNB expansion produce a 336,184-byte L-vector. Inverse BWT and final RLE4 expansion produce the 399,360-byte output.

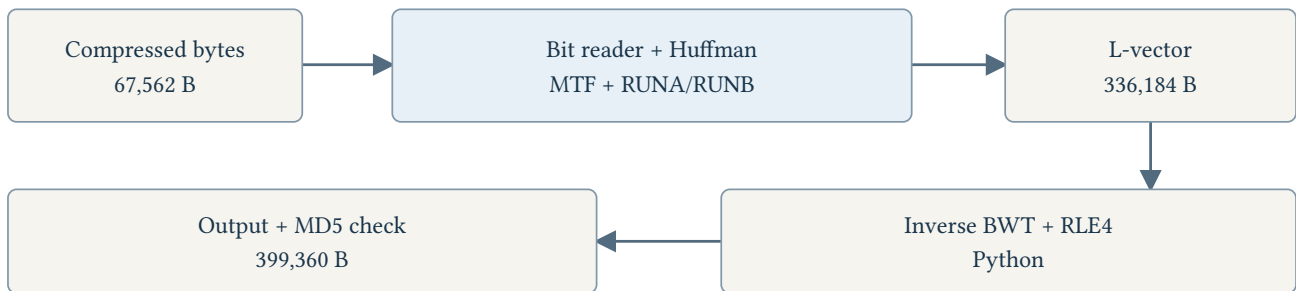


Figure 1: The bzip2 path. RUNA/RUNB and move-to-front are handled together in the symbol loop. The native kernel covers the same combined boundary proposed for hardware.

Configuration	Mean ± SD	vs stock
Stock Python	1,132.50 ± 15.71 ms	1.00×
Optimized Python	288.26 ± 2.71 ms	3.93×

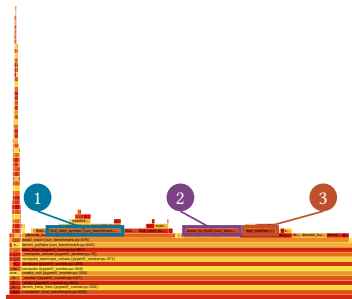
The result is a **74.5% runtime reduction**, exceeding the course's 7% requirement. Output matches `bz2.decompress` byte for byte and passes the unchanged MD5 check. The optimized relative standard deviation is **0.94%**.

Measurement scope. Course QEMU/KVM VM, Ubuntu 22.04, release CPython 3.10.12, pyperformance 1.14.0; 120 measured values per configuration. Source: `results/baseline_pyflate.json` and `optimized_pyflate.json`. Supporting development profiles and ablations use a different environment, labeled on the next page.

2. Profiling: the whole decoding pipeline

The stock Python-frame profile highlights Huffman decoding and bit-buffer work, with move-to-front and inverse BWT alongside them. Development instrumentation found a mean of about five entries visited by the symbol matcher, despite its table capacity of 258. A lookup table can remove this scan, but the measured scan is short; per-symbol interpreter work and the later transformations also deserve attention.

Full recorded flame graph

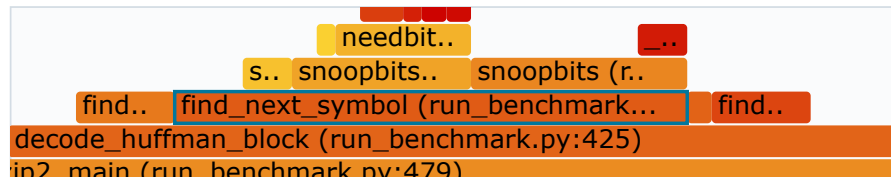


Original widths and stack depth.
Imports and harness retained.
Each outline marks one call path.
The same function may recur.
Percentages are inclusive,
not flat self-time totals.

Highlighted regions - enlarged with surrounding frames

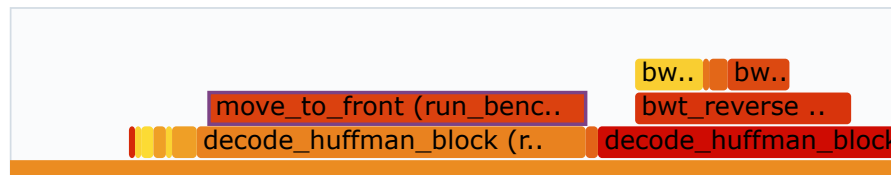
1. Huffman matcher

find_next_symbol | this frame: 20.54% inclusive



2. Alphabet updates

move_to_front | this frame: 15.10% inclusive



3. Inverse BWT

bwt_reverse | this frame: 8.66% inclusive

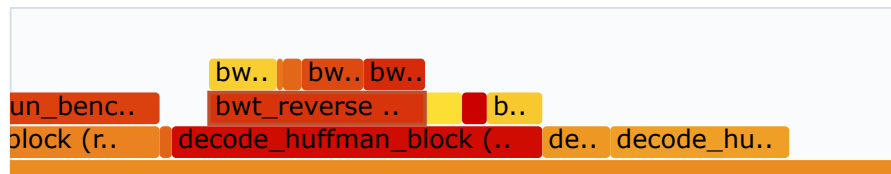


Figure 2: Full stock Python-frame flame graph, retaining startup and harness context. Numbered outlines identify exactly the call paths enlarged on the right. Inclusive percentages use the original whole-profile denominator; nested shares overlap.

A separate **Windows / CPython 3.12.6** cProfile run gives `find_next_symbol` 0.519 s cumulative out of 1.186 s (43.8%), including its bit-reader callees. `decode_huffman_block` has 0.241 s **self** time and 1.172 s **cumulative** time: most of its inclusive cost is in functions it calls. Source: `dev/pyflate/FINDINGS.md §3`.

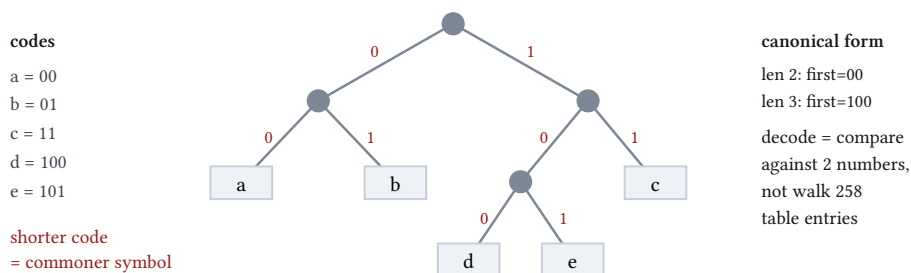


Figure 3: A small Huffman tree explains the optimization boundary: canonical decoding compares a code against length-indexed limits, while the stock matcher walks candidate entries. Shorter codes commonly represent more frequent symbols.

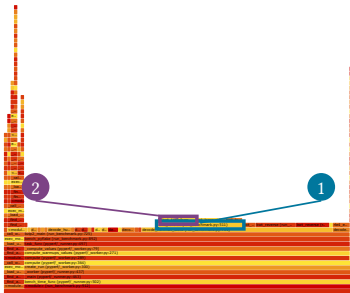
3. Changes that shipped

- **Reduce per-byte overhead:** read the compressed input once, refill the bit window in chunks, bind hot values to locals and accumulate bytes without allocating an object per byte.
- **Decode canonical Huffman codes:** use length-indexed tables and a primary lookup table with a fallback for longer codes. Table construction is amortized over the block.
- **Improve the back end:** replace comparison sorting with counting-sort BWT construction; reverse the MTF list so updates move $O(\text{rank})$ entries; use regex-assisted RLE4 expansion to reduce Python loop iterations. The byte scan/output work is still linear overall.

Component reverted from T3	Runtime	Added cost
None (full T3)	175.5 ms	—
Primary Huffman lookup	185.6 ms	+10.1 ms
Regex-assisted RLE4	207.7 ms	+32.3 ms
Counting-sort BWT	214.0 ms	+38.6 ms

Ablation scope: Windows / CPython 3.12.6, best of seven; dev/pyflate/FINDINGS.md §2b. These are development measurements, not VM timings. Each row reverts one component; the costs need not add. In this experiment, each back-end change contributes more than the primary Huffman table. This motivates optimizing the pipeline rather than the matcher alone.

Full recorded flame graph

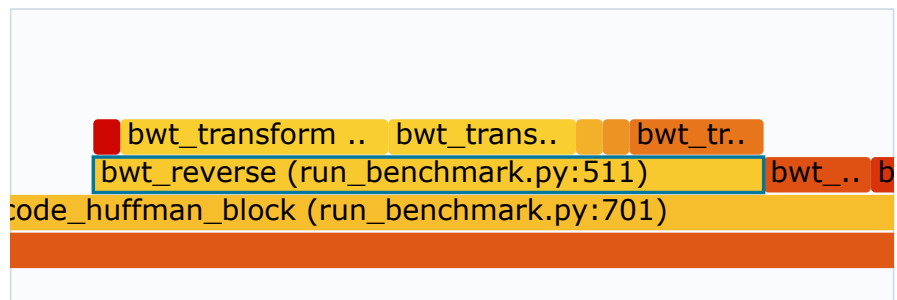


Original widths and stack depth.
Imports and harness retained.
Each outline marks one call path.
The same function may recur.
Percentages are inclusive,
not flat self-time totals.

Highlighted regions - enlarged with surrounding frames

1. Inverse BWT

bwt_reverse | this frame: 24.04% inclusive



2. Index-table construction

bwt_transform | this frame: 9.62% inclusive

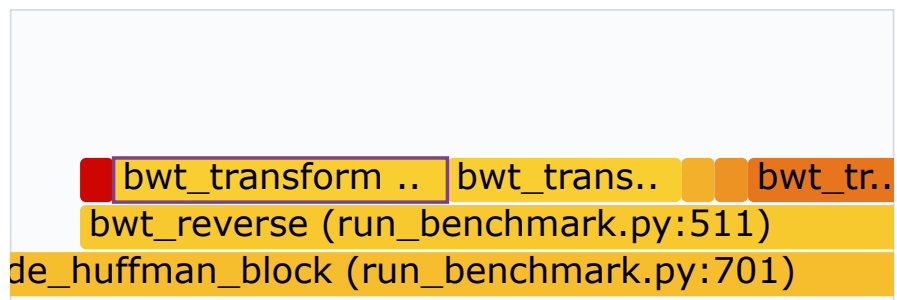


Figure 4: Full optimized **Python** profile, before native offload. Inverse BWT and its index-table construction remain visible. Matcher work was inlined, so its old function frame disappears without implying that Huffman decoding is free.

4. Native execution: what improves, what remains

The Rust/PyO3 BlockDecoder handles the bit reader, Huffman decode, MTF and RUNA/RUNB expansion. Header parsing, inverse BWT, RLE4 and MD5 remain in Python. One block-level call returns the L-vector and the ending bit position; checking both catches output errors and stream desynchronization.

Same benchmark file	Mean $\pm$ SD	vs Python
Python backend	288.55 $\pm$ 5.13 ms	1.00×
Native decode backend	178.12 $\pm$ 5.46 ms	1.62×

These are 120-value VM measurements from `fallback_pyflate.json` and `native_pyflate.json`. They measure the complete benchmark, not the isolated symbol loop. `HWSW_BACKEND=python|native|auto` selects the implementation; native mode requires the wheel, while auto mode falls back. Pyperf workers need `--inherit-environ HWSW_BACKEND`.

Rust is already integrated. A VM dispatch check observed one native decoder call in both native and auto modes, and none in python mode; every output passed MD5. `script_pyflate.sh native` measures this hybrid path. Its optimized stage runs the Python-only course comparison in pyperformance's separate environment.

Matched-work CPU counters

These are medians of three warm-loop VM runs, each decoding the input 16 times, divided by 16. Both backends produced the same output digest. They are a separate experiment from the rigorous timing table above, with acknowledged counter gating after setup (Appendix A2).

Metric per complete decode	Python	Hybrid native
Elapsed time	286.15 ms	174.07 ms
Instructions	1,948.04 M	1,172.25 M
Cycles	670.33 M	404.00 M
IPC	2.91	2.89
Branches	349.36 M	207.14 M
Branch misses (rate)	555,538 (0.159%)	303,573 (0.147%)
Generic cache references	1.673 M	1.593 M
Generic cache misses (rate)	43,252 (2.586%)	42,364 (2.654%)

Native decoding reduces instructions by **39.8%**, cycles by **39.7%** and branch misses by **45.4%**, while IPC is nearly unchanged. The approximately 1.64× speedup in this run comes from removing work, not increasing instruction throughput. Cache misses fall only **2.1%**: the remaining Python BWT/RLE4 pipeline still moves and transforms the block. This is consistent with a partial offload, but aggregate counters cannot attribute misses to a stage. L1-load counts were invalid and are excluded.

Offload limits and the next target

Amdahl's law gives $S = \frac{1}{(1-f) + \frac{f}{s}}$ for kernel speedup s and offloaded fraction f of **optimized Python** runtime. Even infinite symbol-decoder speed leaves BWT and RLE4. Unlike nbody's native integration loop, this offload leaves over a billion instructions per decode in a largely Python pipeline. The next target is the **whole inverse BWT**, including table construction, not just its dependent traversal. Earlier VM phase timings put them at 137.7 ms and 47.8 ms respectively (Appendix A3). A native port and working-set sweep would test whether memory latency then becomes limiting; the present data do not establish that.

5. Hardware acceleration: current proposal

The hardware boundary comprises `huffman_engine` followed by `mtf_cam`. Together they produce the same L-vector as the native decoder. Software keeps block headers, inverse BWT, RLE4 and MD5. A regular decoder datapath and a coarse block-level submission are the main co-design opportunities.

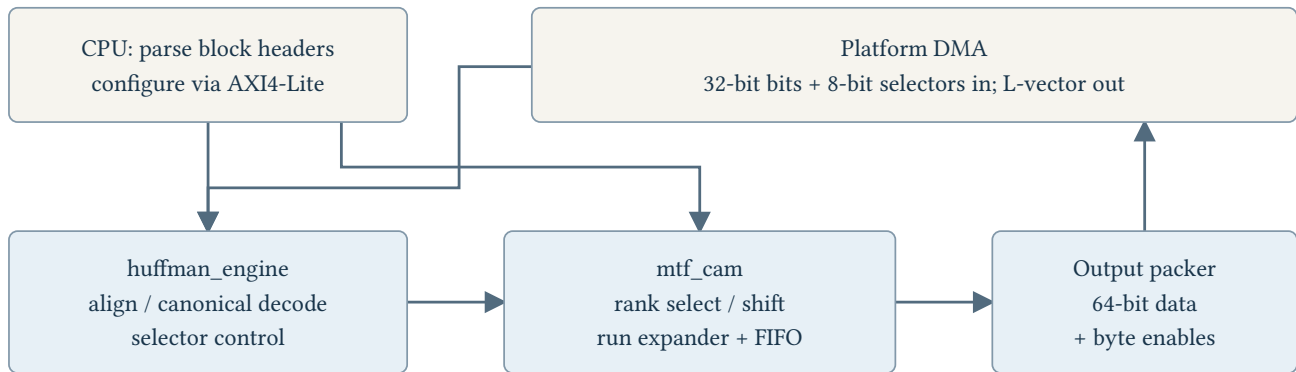


Figure 5: AXI4-Lite configures both modules; AXI4-Stream carries data. The symbol stream stays on chip. Platform DMA returns the L-vector to software.

Huffman engine

A bit aligner feeds a canonical-code comparator/lookup datapath. The selector controller changes the active table every 50 symbols. The current interface specifies a 32-bit compressed input stream, an 8-bit selector stream, and 32-bit output beats containing a 9-bit bzip2 symbol plus type fields. A 32-bit AXI4-Lite window holds controls, `START_BIT`, lengths and counters. It supports up to six tables and a maximum bzip2 code length of 20 bits.

The target is at most 1.10 cycles per symbol, including table construction and switches, with an unstalled benchmark-block model of 153,121 cycles. At the **50 MHz clock target**, that model is 3.06 ms for this module. This is not an achieved frequency or a measured end-to-end hardware runtime.

MTF and run expansion

The input is a **rank**, so the decoder selects an indexed alphabet entry and shifts preceding entries. The current proposal maintains up to 256 entries and expands RUNA/RUNB groups through a buffered output packer. The default output width is eight bytes (64-bit data plus byte enables). It targets one MTF symbol per cycle for any valid rank.

Measured mean rank 7.17 characterizes this input; the one-symbol-per-cycle requirement also covers large ranks. The specified area/throughput sweep varies output width $W = 4, 8, 16$ and FIFO depth. The default module model is 157,560 cycles, or 3.15 ms at 50 MHz. The two modules stream concurrently; their individual modeled latencies cannot simply be added or substituted for an integrated back-pressure simulation.

Implementation status. Requirements and architecture are approved for both modules.

Microarchitecture, RTL, verification, PPA and integration are still open in the recorded hardware status.

Formal MTF invariants are planned, not completed proofs. Drivers and platform DMA remain integration work. No measured area, clock or power benefit is claimed.

6. Conclusion

The Python changes deliver $3.93\times$ over stock with byte-exact output. Native symbol decoding adds $1.62\times$ over the optimized Python backend, leaving substantial work in BWT and RLE4. The hardware proposal follows that measured boundary; its integrated performance and cost remain to be demonstrated.

Evidence: `hw/huffman_engine/docs/{prd,mas}.md`, `hw/mtf_cam/docs/{prd,mas}.md`, and `hw/PROGRESS.md` (2026-09-05). Methods and supporting profiles: **report_appendix.pdf**. Repository: [HWSW_Project](#).